

cia-1

June 27, 2026

Name: Rahul Gupta

Register Number: 2547242

Class: 4 MCA B

Subject: Quantum Computing

CIA - 1

Q.1. Design the circuit using qiskit and check the output for different input combinations

```
[ ]: # Import libraries

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer.primitives import Sampler
from qiskit.visualization import plot_histogram
import numpy as np
from qiskit.quantum_info import Statevector
```

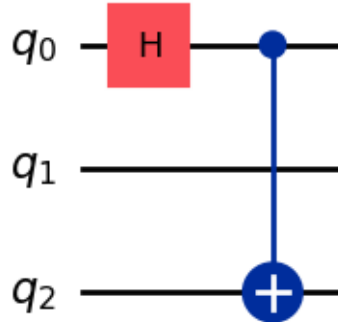
```
[ ]: # Circuit design using qiskit

# Create base circuit with 3 qubits - q0, q1, q2
circuit = QuantumCircuit(3)

# Add hadamard gate to q0
circuit.h(0)

# add controlled not gate to q2
# Qubit 0 (q0) is the control, qubit 2 (q2) is the target
circuit.cx(0, 2)

display(circuit.draw(output='mpl'))
```



```
[ ]: # Check the output for different input combinations

# List of all 8 possible 3-qubit inputs
inputs = ['000', '001', '010', '011', '100', '101', '110', '111']

for i in inputs:
    # Create a base circuit initialized to the specific input bitstring
    circuit = QuantumCircuit(3)
    # from_label() sets up the starting state
    initial_state = Statevector.from_label(i)

    # Append circuit design
    circuit.h(0)
    circuit.cx(0, 2)

    # Evolve the initial state vector through your circuit
    output_state = initial_state.evolve(circuit)

    # Print the result clearly
    print(f"Input State: |{i}>")
    print("Output State:")
    display(output_state.draw('latex'))
```

Input State: |000>

Output State:

$$\frac{\sqrt{2}}{2}|000\rangle + \frac{\sqrt{2}}{2}|101\rangle$$

Input State: |001>

Output State:

$$\frac{\sqrt{2}}{2}|000\rangle - \frac{\sqrt{2}}{2}|101\rangle$$

Input State: $|010\rangle$

Output State:

$$\frac{\sqrt{2}}{2}|010\rangle + \frac{\sqrt{2}}{2}|111\rangle$$

Input State: $|011\rangle$

Output State:

$$\frac{\sqrt{2}}{2}|010\rangle - \frac{\sqrt{2}}{2}|111\rangle$$

Input State: $|100\rangle$

Output State:

$$\frac{\sqrt{2}}{2}|001\rangle + \frac{\sqrt{2}}{2}|100\rangle$$

Input State: $|101\rangle$

Output State:

$$-\frac{\sqrt{2}}{2}|001\rangle + \frac{\sqrt{2}}{2}|100\rangle$$

Input State: $|110\rangle$

Output State:

$$\frac{\sqrt{2}}{2}|011\rangle + \frac{\sqrt{2}}{2}|110\rangle$$

Input State: $|111\rangle$

Output State:

$$-\frac{\sqrt{2}}{2}|011\rangle + \frac{\sqrt{2}}{2}|110\rangle$$

We test different starting inputs by using a loop that initializes the qubits to states like $|000\rangle$ or $|111\rangle$.

The code simulates how these initial inputs change step-by-step through the circuit to reveal every possible outcome.

Q.2.

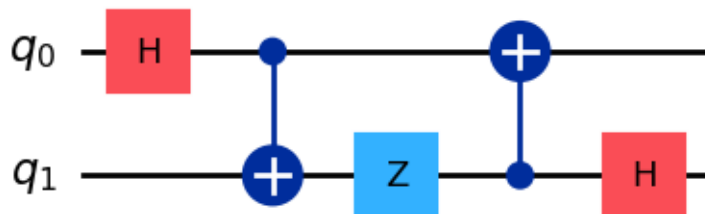
- a) Compute the output of the 2 qubits quantum circuit

```
[ ]: # Circuit design using qiskit

circuit = QuantumCircuit(2)

# Append gates
circuit.h(0)
circuit.cx(0, 1)
circuit.z(1)
circuit.cx(1, 0)
circuit.h(1)

# Display circuit
display(circuit.draw(output='mpl'))
```



```
[ ]: # compute the output

# Setup circuit
circuit = QuantumCircuit(2)

# append circuit designs
circuit.h(0)
circuit.cx(0, 1)
circuit.z(1)
circuit.cx(1, 0)
circuit.h(1)

# Define initial state: q1 = |1>, q0 = |0> -> string label '10'
initial_state = Statevector.from_label('10')

# Evolve the state through the circuit
final_state = initial_state.evolve(circuit)

print("Calculated Final State Vector:")
display(final_state.draw('latex'))
```

Calculated Final State Vector:

$$|11\rangle$$

Starting from the $|10\rangle$ state, each gate mathematically changes the qubits through superpositions, entanglements, and phase-flips.

After tracking every gate in sequence, the circuit forces both qubits to end deterministically in the $|11\rangle$ state.

Q.2.

- b) Proof that the order of unitary compositions is crucial in quantum operations considering the combination of Hadamard, phase gate and T-gate

```
[21]: # Circuit 1: Sequence A (H -> S -> T)
qc_a = QuantumCircuit(1)
qc_a.h(0) # Hadamard gate
qc_a.s(0) # Phase gate
qc_a.t(0) # T gate

# Circuit 2: Sequence B (T -> S -> H)
qc_b = QuantumCircuit(1)
qc_b.t(0) # T gate
qc_b.s(0) # Phase gate
qc_b.h(0) # Hadamard Gate

# Calculate final outputs starting from the |0> state
state_initial = Statevector.from_label('0')
state_a = state_initial.evolve(qc_a)
state_b = state_initial.evolve(qc_b)

# Display the resulting different state vectors
print("Output of Sequence A (H -> S -> T):")
display(state_a.draw('latex'))

print("\nOutput of Sequence B (T -> S -> H):")
display(state_b.draw('latex'))
```

Output of Sequence A (H -> S -> T):

$$\frac{\sqrt{2}}{2}|0\rangle + \left(-\frac{1}{2} + \frac{i}{2}\right)|1\rangle$$

Output of Sequence B (T -> S -> H):

$$\frac{\sqrt{2}}{2}|0\rangle + \frac{\sqrt{2}}{2}|1\rangle$$

Quantum gates are represented by matrices, and order matters because matrix multiplication is non-commutative (changing the order changes the final result).

Applying a Hadamard gate followed by a T-gate produces a completely different final quantum state than applying the T-gate first and the Hadamard gate second.
